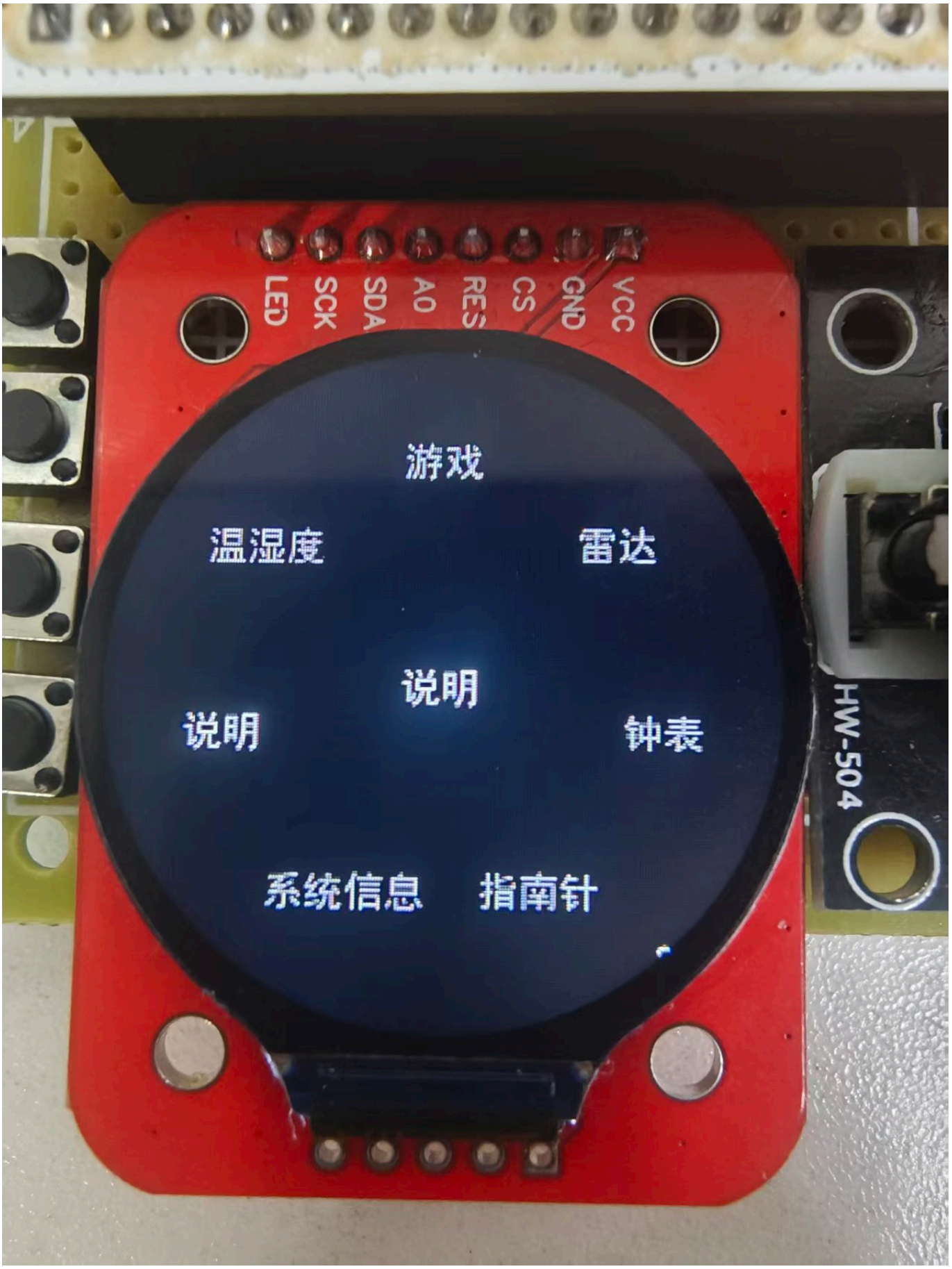
MYCLASS_MX —— STM32F407 圆屏多功能掌机

嵌入式系统课程设计项目。以 STM32F407VGT6 为平台，在 240×240 圆形 IPS 屏上实现了一套 **带环形菜单、传感器仪表和 19 个可玩小游戏** 的图形化交互终端。

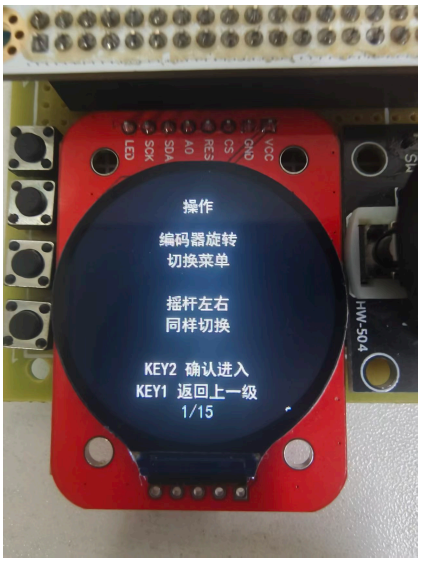


1. 项目概览

项目	内容
主控	STM32F407VGT6（Cortex-M4F，168 MHz，1 MB Flash / 192 KB SRAM）
显示	GC9A01 1.28" 240×240 圆屏，SPI 接口，RGB565
图形库	LVGL 8.3.11（自定义中文 14px 字模，圆角/圆形控件适配圆屏）
操作系统	FreeRTOS（CMSIS-OS2 封装），4 个任务
输入	4 个独立按键 + 旋转编码器（带按压）+ 双轴模拟摇杆（带按压）
传感器	DHT11 温湿度、HC-SR04 超声波、HMC5883L 三轴磁力计
声光	有源蜂鸣器 + 6 颗 LED（生命值 / 流水灯提示）
工具链	STM32CubeMX 生成底层 + Keil MDK（AC5）编译

功能一览：圆环主菜单（游戏 / 雷达 / 钟表 / 指南针 / 系统信息 / 说明 / 温湿度）、 游戏子菜单 19 款小游戏、声光提示服务、中文分页说明文档。

目录：1. 项目概览 | 2. 硬件基础 | 3. 软件架构 | 4. 单片机知识点 | 5. 应用层 | 6. 编译与运行 | 7. 已知限制 | 8. 相关文档

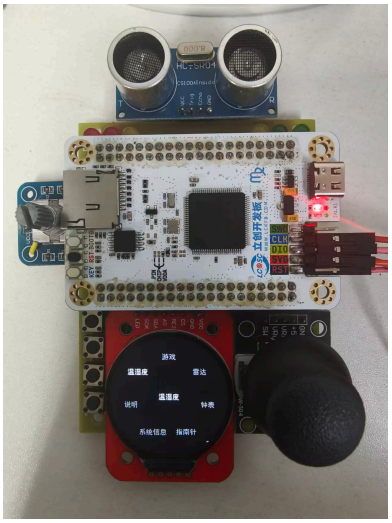
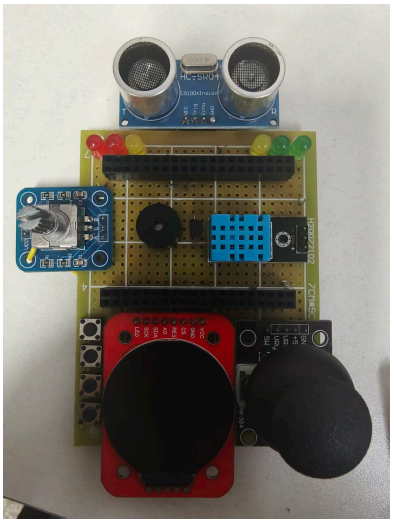
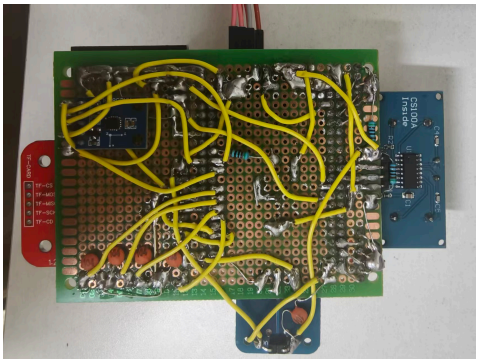
系统信息	说明	游戏菜单
		

2. 硬件基础

2.1 主控与最小系统

- **STM32F407VGT6**：Cortex-M4F 内核，单精度硬件 FPU + DSP 指令，最高 168 MHz； 1 MB Flash、192 KB SRAM（其中 64 KB CCM 只能被 CPU 访问，**DMA 不可用**），LQFP100 封装。
- **时钟源**：外部 8 MHz 晶振（HSE）作为 PLL 输入，最终 168 MHz 系统时钟（见 4.1）。
- **下载调试**：SWD（SWDIO/SWCLK）两线调试；板载 RESET、BOOT0、KEY 按键。
- **供电**：3.3 V 逻辑，USB 供电；外设模块（超声波、磁力计、温湿度）均为 3.3/5 V 兼容。

2.2 板卡与外设实物

正面（带芯片）	正面（去芯片）	背面
		

正面：STM32F407 核心板（TF 卡槽、USB、彩色排针、晶振）+ 圆形 GC9A01 屏（模块丝印 `LED SCK SDA A0 RES CS GND VCC`）+ 五针模拟摇杆 + HC-SR04 超声波模块。

背面：洞洞板走线，固定了 HMC5883L 磁力计模块（I²C）、TF 卡转接板等。

> TF 卡槽与 TF 卡模块在本工程中**未使用**（未启用 SDIO/FATFS），仅作为硬件资源保留。`DEMO/` 目录是开发过程中的单外设验证例程，已被 `APP/` 的正式应用取代。

2.3 引脚分配

功能	引脚	配置	备注
LCD 背光	PA2 `LCD_LED`	推挽输出	高电平点亮
LCD SPI 时钟	PA5 `LCD_SCK`	复用 AF5，非常高速	SPI1_SCK
LCD SPI 数据	PA7 `LCD_SDA`	复用 AF5，非常高速	SPI1_MOSI（1-line 单向）
LCD 数据/命令	PE7 `LCD_A0`	推挽输出	1=数据，0=命令（DC）
LCD 复位	PE9 `LCD_RES`	推挽输出	低电平复位
LCD 片选	PE11 `LCD_CS`	推挽输出	低有效，DMA 完成后拉高
编码器 A 相	PA1 `ENCODER_A`	复用 AF2，上拉	TIM5_CH2
编码器 B 相	PA0 `ENCODER_B`	复用 AF2，上拉	TIM5_CH1
编码器按压	PD3 `ENCODER_KEY`	上拉输入	低有效
按键 KEY1	PC5	上拉输入	全局“返回上一级”
按键 KEY2	PB1	上拉输入	应用确认/开火/下钩…
按键 KEY3	PE8	上拉输入	上一项 / 减速
按键 KEY4	PE10	上拉输入	下一项 / 加速

功能	引脚	配置	备注
摇杆 X	PC3 `JS_VRx`	模拟	ADC1_IN13
摇杆 Y	PC1 `JS_VRy`	模拟	ADC1_IN11
摇杆按压	PD9 `JS_KEY`	上拉输入	低有效
超声波 Trig	PB9 `US_Trig`	推挽输出	≥10 μs 高脉冲
超声波 Echo	PB7 `US_Echo`	复用 AF2	TIM4_CH2 输入捕获
温湿度 DHT11	PC6 `DHT11_DATA`	开漏输出 + 内部上拉	单总线，主机可读 IDR
磁力计 SCL	PB10 `MAG_SCL`	复用 AF4 开漏	I2C2_SCL
磁力计 SDA	PB11 `MAG_SDA`	复用 AF4 开漏	I2C2_SDA
磁力计 DRDY	PE14 `MAG_READY`	浮空输入	低电平=数据就绪
蜂鸣器	PB12 `BEEP_Y`	推挽输出	有源蜂鸣器，高电平响
LED1~LED6	PC9 / PC12 / PC11 / PD0 / PA8 / PC7	推挽输出	生命值、流水灯

3. 软件架构

```
┌──────────────────────────────────────────────────────────────┐
│ APP 层        app_launcher（菜单/生命周期）                  │
│               app_radar/clock/compass/info/help/temp + 19 games│
│               sys_notify（声光服务） sys_sensor（传感器汇总）  │
├──────────────────────────────────────────────────────────────┤
│ 中间件        LVGL 8.3.11（控件/绘制/字模/定时器）           │
│               FreeRTOS（任务/队列/时基）                     │
├──────────────────────────────────────────────────────────────┤
│ 驱动层        HARDWARE: gc9a01 / lcd_spi / keys / encoder /  │
│               joystick / beep / led / dht11 / hcsr04 / hmc5883l│
│               LVGL_BSP: lv_port_disp / lv_port_indev         │
├──────────────────────────────────────────────────────────────┤
│ HAL/CMSIS     STM32F4 HAL（GPIO/SPI/DMA/ADC/I2C/TIM）        │
└──────────────────────────────────────────────────────────────┘
```

目录	说明
`Core/`	CubeMX 生成的时钟、GPIO、SPI、ADC、I2C、TIM、FreeRTOS 初始化
`HARDWARE/`	板级外设驱动（与具体引脚解耦的 API）
`LVGL_BSP/`	LVGL 显示/输入移植层、中文字模 `lv_font_cn_14.c`
`APP/`	应用与游戏（启动器框架 + 业务逻辑）

目录	说明
DEMO/	早期单外设验证例程（保留参考）
DOC/	截图、中文字模生成指南、参考驱动与算法
MDK-ARM/	Keil 工程（MYCLASS_MX.uvprojx）

4. 涉及的单片机知识点

4.1 时钟树与 PLL

Core/Src/main.c 的 SystemClock_Config()：

- HSE = 8 MHz → PLLM = 4 → 2 MHz 比较频率 → PLLN = 168 → VCO 336 MHz → PLLP = 2 → **SYSCLK = 168 MHz**，PLLQ = 4 → 84 MHz（USB/SDIO 用）。
- AHB 不分频（168 MHz），APB1 四分频（**42 MHz**），APB2 二分频（**84 MHz**）。
- PWR_REGULATOR_VOLTAGE_SCALE1 + FLASH_LATENCY_5：168 MHz 下必须提高内核电压等级并插入 5 个 Flash 等待周期，否则取指错误。

注意：APB1 上的定时器在分频系数≠1 时，定时器时钟会被硬件 ×2，因此 TIM4/TIM5 实际 计数时钟是 **84 MHz**（这一点直接决定了下面两个定时器的分频值）。

4.2 GPIO 的四种用法

- **推挽输出**：LCD 背光/复位/片选/DC、蜂鸣器、6 颗 LED。
- **上拉输入**：4 个按键、编码器按压、摇杆按压——按键接地，按下读到低电平。
- **复用推挽**：SPI1 的 SCK/MOSI（GPIO_MODE_AF_PP + GPIO_AF5_SPI1）； 定时器输入捕获 PB7（GPIO_AF2_TIM4）。
- **复用开漏**：I2C2 的 SCL/SDA（GPIO_MODE_AF_OD + GPIO_AF4_I2C2），依赖外部上拉； DHT11 数据线用**开漏输出 + 内部上拉**，写 1 即释放总线，同时 IDR 仍能读出真实电平。

4.3 按键消抖与“按下沿”事件

HARDWARE/keys.c 不是简单阻塞延时，而是**基于时间戳的状态机**：

1. keys_update() 采样 6 个键，电平变化时记录 change_tick；
2. 电平稳定超过 KEYS_DEBOUNCE_MS = 20 ms 才更新“稳定状态”；
3. 稳定状态由 0→1 时置位 edges 位图；keys_scan() 返回并清除该位图，实现**按下沿一次性 事件**，避免长按重复触发。

编码器按压键在 keys.c 中覆盖了 lv_port_indev.c 的弱函数 lv_port_encoder_key()，保证 “物理按键”和 “LVGL 输入设备”读到的是同一份消抖后的状态。

4.4 旋转编码器：定时器编码器接口（TIM5）

Core/Src/tim.c：TIM5 工作在 TIM_ENCODERMODE_TI12（A/B 两相**四倍频**计数）， 32 位计数范围（Period = 0xFFFFFFFF），输入滤波器 ICFilter = 8（抑制旋转抖动）。

HARDWARE/encoder.c 用**无符号做差再转有符号**读取增量：delta = (int32_t)(now - last)， 计数器 32 位回绕时结果依然正确。上位逻辑再按 ENC_DETENT = 4 换算成“档”（一格）， 菜单里一格 = 移动一项。

4.5 输入捕获测距（TIM4 + HC-SR04）

`HARDWARE/hcsr04.c`：

- 分频 83 → 计数时钟 1 MHz，**每计数 = 1 µs**（ARR = 65535，16 位）。
- 流程：配置上升沿捕获 → 清标志/清计数器 → 启动捕获 → 发 12 µs Trig 脉冲 → 轮询 `TIM_FLAG_CC2` 等上升沿 → 改成下降沿捕获 → 再等一次 → `t_end - t_start` 即回波宽度。
- 换算：`距离(cm) = 脉宽(µs) / 58`（声速 343 m/s，往返各一次）。
- 超时用 **DWT 周期计数器**（`DWT->CYCCNT`）而不是 SysTick，因此可以在任意任务中安全使用， 不会被调度器打断逻辑。

4.6 ADC + DMA：双轴摇杆

`Core/Src/adc.c`：ADC1 双通道**规则组扫描**（IN11 / IN13），12 位分辨率、连续转换、 采样时间 84 周期，DMA 循环模式（`DMA2_Stream0 / Channel 0`，半字对齐）， 转换结果直接写入 `joy_adc_buf[2]`，CPU 无需参与。

`HARDWARE/joystick.c` 再做了三件事：

1. **上电自校准**：连续 16 次采样求中点，避免电位器个体差异；
2. **归一化**：`(raw - center) * 1000 / 2048` → −1000~+1000；
3. **带阈值的方向判定**：X/Y 独立比较阈值得到位掩码，支持 8 方向；并提供 `JOY_XY_SWAP` 等宏修正装配方向。

4.7 I²C 读写磁力计（HMC5883L）

`HARDWARE/hmc5883l.c`：I2C2 100 kHz、7 位地址 `0x1E`（HAL 需左移成 8 位）。

- 上电先读 ID 寄存器（`0x0A` 起 3 字节，应为 `H 4 3`）做器件自检；
- 配置 A = `0x70`（8 次平均、15 Hz、正常测量），配置 B = `0x20`（增益 1090 LSB/Gauss， ±1.3 Ga），模式 = `0x00`（连续测量）；
- 数据寄存器顺序是 **X, Z, Y**（不是 X, Y, Z），读 6 字节后按高低字节拼接；
- 航向角：`heading = atan2f(x, y) * 180/π`，再叠加磁偏角补偿 （宏 `HMC5883L_MAG_DECL_DEG10`，按当地磁偏角调整）并归一化到 0~360°；
- 校准采用 **min-max 法**：全方位旋转记录各轴极值，中值作"硬铁偏移"，半幅作比例因子， 旋转不充分（半幅 < 100 LSB）时拒绝生效。

4.8 单总线时序（DHT11）

`HARDWARE/dht11.c` 用 DWT 实现微秒级时序（不开定时器也能量 µs）：

- 主机拉低 ≥18 ms → 释放 30 µs；
- 等待传感器应答（低 80 µs / 高 80 µs / 再低）；
- 连续读 40 位：每位先 50 µs 低电平，再判高电平宽度——**>40 µs 为 1，否则为 0**；
- 5 字节中前 4 字节之和的低 8 位等于第 5 字节才算校验通过。

4.9 DMA + SPI 刷屏

`Core/Src/spi.c`：SPI1 主机、1-line 单向、8 位、`CPOL=0 / CPHA=1Edge`（模式 0）、软件 NSS， 预分频 4 → **21 MHz**（APB2 = 84 MHz）。

`HARDWARE/lcd_spi.c` 用 `HAL_SPI_Transmit_DMA` 发送像素流，DMA = `DMA2_Stream3 / Channel 3`， 内存→外设、字节对齐、普通模式；传输完成中断里 `HAL_SPI_TxCpltCallback()` 拉高 CS 并 `lv_disp_flush_ready()` 通知 LVGL 这块缓冲已刷完。

> 关键约束：**DMA 只能访问主 SRAM**，所以两块显示缓冲必须放在普通 SRAM，不能放 CCM。

4.10 FreeRTOS 任务划分

`Core/Src/freertos.c`：

任务	栈	优先级	职责
`LVGL_TASK`	4 KB	Normal7（最高）	`lv_init` → 显示/输入移植 → 启动器，循环 `lv_tick_inc(5)` + `lv_timer_handler()` + `osDelay(5)`
`defaultTask`	512 B	Normal	空转占位
`sensor_task`	1 KB	Low	`sys_sensor_poll()` 每 20 ms 一轮：DHT11（1 s）、HC-SR04（250 ms，游戏中 70 ms）
`notify_task`	512 B	Low1	从消息队列取声光命令并播放

设计要点：

- **阻塞型传感器隔离**：DHT11 起始信号要拉低 18 ms、超声波要等回波，若放在 UI 任务里会卡住 刷新；因此放进独立低优先级任务，结果写入全局 `g_sensor`，UI 只读快照。
- **时基**：`LV_TICK_CUSTOM = 0`，由 LVGL 任务每 5 ms 手动 `lv_tick_inc(5)`，避免与 SysTick/FreeRTOS 时基冲突。
- **声光服务化**：LED 与蜂鸣器由 `notify_task` 独占，其他任务只发命令（`osMessageQueueNew(8, ...)`）， 避免多任务同时操作同一 GPIO 造成“串音/闪灯错乱”。

4.11 LVGL 移植：显示与输入

- 显示：**双部分缓冲**（每块 240×40 像素 = 19.2 KB，两块 38.4 KB）， `flush_cb` 里设窗口（`CASET/RASET/RAMWR` 由 `gc9a01_set_window()` 完成）后启动 DMA， 渲染与传输可并行；`LV_COLOR_DEPTH = 16`（RGB565）。
- 输入：注册 **编码器**（`encoder_delta()` → `LV_INDEV_STATE_PRESSED/RELEASED` 滚动）、 **按键**（KEY2/KEY3/KEY4/摇杆 → `LV_KEY_ENTER/PREV/NEXT/ESC`）、 **摇杆**（方向 → `LV_KEY_UP/DOWN/LEFT/RIGHT`）。
- 内存：`LV_MEM_SIZE = 24 KB`，因此应用层大量采用“**整屏 canvas 自绘**”策略（见 4.12）。

4.12 圆屏 UI 的两条硬约束

1. **可视区**：240×240 圆屏圆心 (120,120)，四角不可见，可用内接正方形约 (35,35)–(205,205)。 所有游戏/仪表的活动区都限制在该范围内。
2. **对象数**：LVGL 每个对象都有额外开销，24 KB 堆顶不住几十个控件。 游戏统一使用 **1 个 `lv_obj_create` 全屏对象 + `LV_EVENT_DRAW_MAIN` 里用 `lv_draw_rect()` / `lv_draw_line()` 直接绘制**，只用 1~2 个 `lv_label` 显示分数， 退出时 `lv_obj_clean()` 全部回收。

4.13 中文字模（4bpp 亚像素 + 压缩 + 稀疏表）

`LVGL_BSP/lv_font_cn_14.c` 由在线字体转换工具生成：

- `--bpp 4 --size 14 --lcd --use-color-info --range 32-127`，即 ASCII 用 range 0（95 字），中文用**稀疏 cmap**（`unicode_list` 存 `codepoint - 176`，`list_length = 296`）；
- `--lcd` 生成亚像素字模，需 `LV_USE_FONT_SUBPX = 1`；压缩位图需 `LV_USE_FONT_COMPRESSED = 1`；`LV_FONT_SUBPX_BGR` 控制红蓝彩边（0 = RGB）；
- `LV_FONT_DEFAULT = &lv_font_cn_14`，全局默认中文字体，缺字会显示方块（见 `DOC/中文字模生成指南.md`）。

4.14 资源占用

项目	数值
Flash（代码 + 只读数据）	约 450 kB / 1 MB（其中中文字模占大头）
SRAM（RW + ZI）	约 104 kB / 128 kB（两块显示缓冲 38.4 kB、LVGL 堆 24 kB）
LVGL 刷新周期	30 ms（`LV_DISP_DEF_REFR_PERIOD`）

---

5. 应用层：菜单框架与 25 个应用

> 应用层代码全部位于 `APP/`，每个应用 = 一对 `.c/.h` + 一个 `const app_t` 实例。

5.1 启动器框架

`APP/app_launcher.h` 定义统一生命周期接口：

```
typedef struct {
    const char *name;   /* 菜单显示名（中文） */
    void (*enter)(void);/* 进入：建对象 + 建 lv_timer */
    void (*exit)(void); /* 退出：删自己的 lv_timer */
} app_t;
```

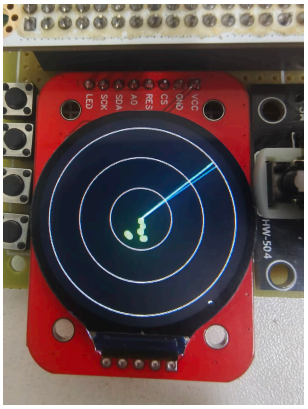
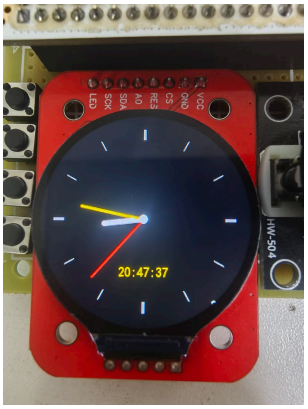
- `app_register()` 注册主菜单项，`app_register_game()` 注册游戏子菜单项；
- 菜单是**半径 80 的圆环**，中心大字显示当前选中项；
- 条目 ≤ 7 时按 `360°/n` 均匀排布；> 7 时切换为 **7 个固定槽位（间隔 36°）的滚动窗口**， 只更新文字，避免 19 个游戏挤成一圈；
- 输入约定：编码器旋转 / 摇杆左右 = 切换，`ENCODER_KEY` / `JS_KEY` = 确认， **KEY1 归启动器所有 = 返回上一级**（应用不得消费），切换与确认都触发 `NOTIFY_MARQUEE_ONCE` 做声光反馈。

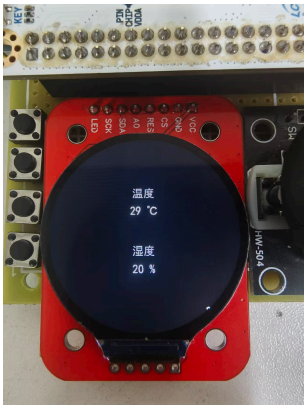
5.2 主菜单（6 项 + 游戏入口）

应用	功能	交互
雷达 `app_radar`	超声波雷达：扫描线旋转，障碍按"距离=半径"显示为亮点并渐隐	自动刷新

应用	功能	交互
钟表 app_clock	模拟表盘 + 数字时间，可调时/分/秒	编码器调值，ENC/JS 切字段
指南针 app_compass	指针固定朝上、刻度盘随方位旋转，正北方向蜂鸣提醒	KEY2 开始/结束校准
系统信息 app_info	运行时长、LVGL 内存占用、**音效开关**	KEY2 切换音效
说明 app_help	15 页中文操作说明（操作/指南针/3 个传感器游戏/9 个游戏/总览/音效）	KEY4·摇杆右·编码器顺时针翻页
温湿度 app_temp	显示 DHT11 温湿度	自动刷新

雷达 **钟表** **温湿度** **指南针**





5.3 游戏子菜单（19 款）

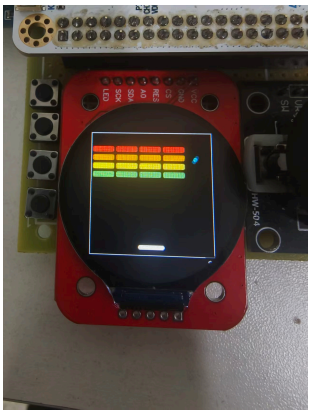
#	游戏	玩法（均做了“易上手”简化）	主要操作
1	贪吃蛇	17×17 网格，撞墙/撞自己扣命（6 命）	摇杆方向，ENC/JS 开始
2	打砖块	挡板接球打砖，丢球扣命	编码器/摇杆（比例控制），KEY2 发球
3	2048	4×4 合并数字	摇杆四向 / 编码器左右
4	数字华容道	4×4 滑块拼回 1~15	摇杆/编码器方向
5	五子棋	13×13 双人轮流	摇杆移动光标，ENC/JS 落子
6	打豆豆	8×8 同色相连（≥2）消除	摇杆/编码器移动光标，ENC/JS 消除
7	俄罗斯方块	10×20，旋转/加速/落底	摇杆左右、下加速、ENC 旋转、KEY2 落底
8	扫雷	8×8 / 10 雷，首翻格及 8 邻域无雷	摇杆/编码器移动，ENC/JS 翻开，KEY2 插旗

#	游戏	玩法（均做了“易上手”简化）	主要操作
9	小恐龙跳跃	**超声波测距控制跳跃**：手靠近 ≤12 cm 起跳，退到 ≥20 cm 才能再跳	手靠近传感器（KEY2 备用）
10	打地鼠	**超声波测距控制锤子位置**：4 cm 最左、40 cm 最右，盖住地鼠自动敲中	手移动 / 摇杆 / 编码器 / KEY3·KEY4
11	大鱼吃小鱼	吃小鱼变大，碰大鱼扣命	摇杆/编码器左右，KEY2 开始
12	坦克射击	KEY2 开炮击毁上方敌方坦克，中弹扣命	摇杆/编码器移动，KEY2 开炮
13	黄金矿工	矿钩摆动，下钩抓金块，限时挖完刷新一轮	KEY2 / JS / ENC 下钩
14	切水果	移动刀锋挥刀切水果，切炸弹扣命	摇杆/编码器移动，KEY2 挥刀
15	飞行射击	战机自动开火，击落俯冲敌机	摇杆/编码器移动
16	滑雪大冒险	雪道下滚，左右躲树，越跑越快	摇杆/编码器左右
17	台球	1 白球 + 3 彩球，瞄准后击球，进洞加分	摇杆/KEY3·KEY4 调角，KEY2 击球
18	地铁跑酷	三赛道，换道躲障碍	摇杆左右 / KEY3·KEY4 换道
19	狙击外星人	移动准星开枪，外星人跑掉扣命	摇杆四方向/编码器移动，KEY2 开枪

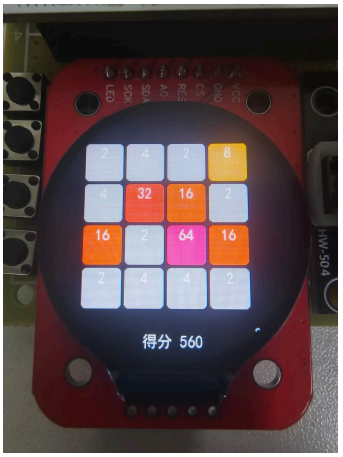
贪吃蛇



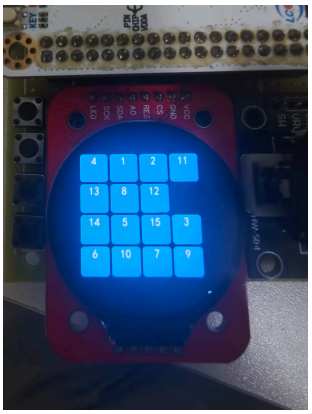
打砖块



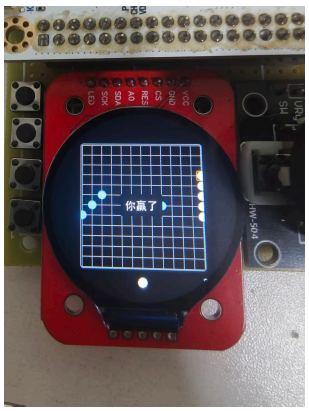
2048



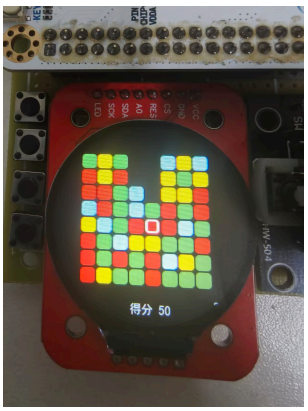
数字华容道



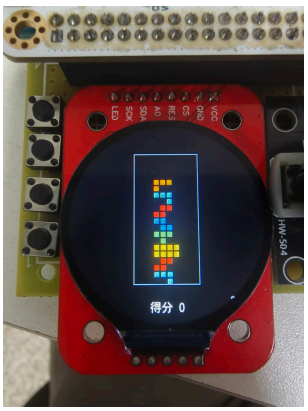
五子棋



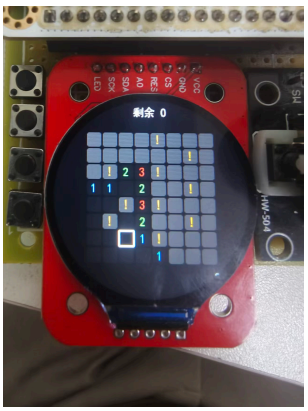
打豆豆

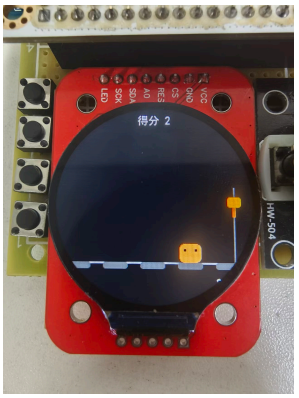
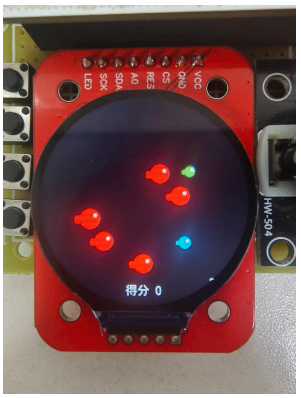
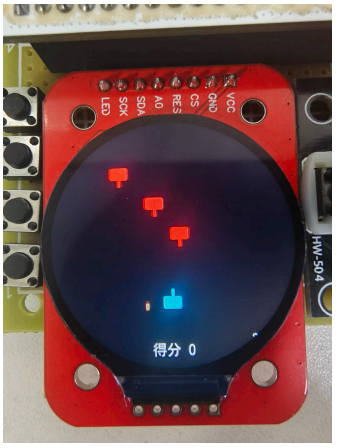


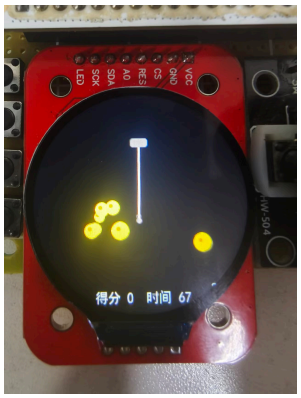
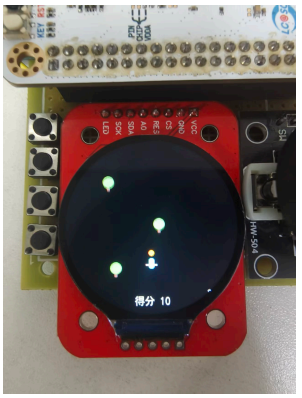
俄罗斯方块

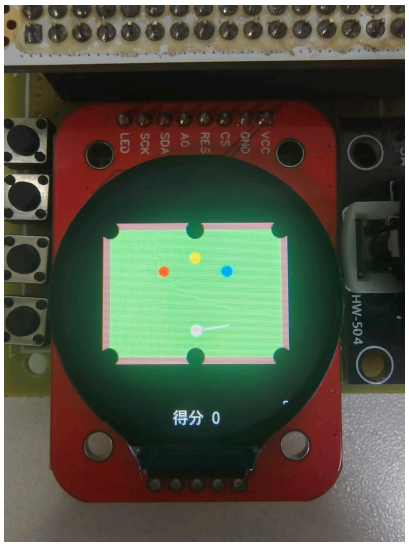
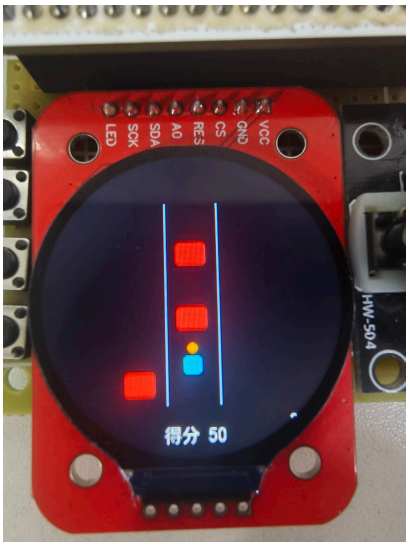


扫雷



跳跃小恐龙	打地鼠	大鱼吃小鱼	坦克射击（截图名“坦克动荡”）
			

黄金矿工	切水果	飞行射击（截图名“雷霆战机”）	滑雪大冒险
			

台球	地铁跑酷	狙击外星人
		

游戏层公共设计

- 每个游戏 1 个 `lv_timer`（20~30 ms 一拍）驱动“逻辑 + 重绘”，退出时删除；
- 动作类游戏有生命值（新做的 9 款为 3 命，早期贪吃蛇/打砖块为 6 命），用 `game_common` 的 `game_lives_set/dec` 点亮 6 颗 LED 中的前 n 颗，扣命时蜂鸣提醒；`game_lives_clear()` 在退出时熄

灭； 棋盘类游戏则以分数/胜负为目标，结束后按 ENC/JS/KEY2 重开；
- 撞机/撞障碍后给 20 拍左右**无敌时间**，避免一次碰撞连扣多条命；
- 所有绘制都在 `LV_EVENT_DRAW_MAIN` 里做，坐标限制在圆屏内接正方形内。

5.4 声光通知服务

`APP/sys_notify.h`：8 种命令（短鸣、长鸣、成功、错误、流水灯、生命值、单次流水灯、停止）， 由 `notify_send()` 投递到队列，`notify_task` 异步执行；**新命令立即打断旧效果**。 音效总开关 `sys_notify_sound_set()` 默认开启，关闭后只做灯光不出声（不掉电保存）。

5.5 传感器数据分发

`APP/sys_sensor.c` 统一在 `sensor_task` 中轮询，写入 `g_sensor`（温度/湿度/距离/时间戳/有效标志）：

- DHT11 每 1 s 一次（器件最小间隔）；
- HC-SR04 常规 250 ms 一次，进入小恐龙/打地鼠时通过 `sys_sensor_set_fast(1)` 切到 70 ms 快采样（HC-SR04 建议 ≥60 ms，避免上一发回波串扰）；
- 应用层（雷达/温湿度/小恐龙/打地鼠）只读 `g_sensor`，**不直接操作硬件**。

---

6. 编译与运行

1. 用 Keil MDK 打开 `MDK-ARM/MYCLASS_MX.uvprojx`，目标 `MYCLASS_MX`，编译器 **AC5**，C99 模式；
2. 直接 Build → Download 到板子；
3. 上电后进入圆环菜单，`KEY1` 随时返回上一级。

注意事项：

- 含中文的 `.c/.h` 必须保存为 **UTF-8 with BOM**，否则 AC5 按 GBK 解析会出现 `missing closing quote`；
- `lv_conf.h` 中 `LV_USE_FONT_COMPRESSED`、`LV_USE_FONT_SUBPX` 必须为 1，否则文字不显示；
- 字模只能整颗重新生成替换，**不要手工修改**字模表（三张表必须严格对齐）。

---

7. 已知限制与可扩展方向

- 圆屏四角不可见，界面设计需留安全边距；
- LVGL 堆仅 24 KB，复杂界面需改为 canvas 自绘或增大 `LV_MEM_SIZE`；
- 字模为固定 296 个中文，新增文案需同步重新生成字模；
- TF 卡、USB、串口等外设尚未启用，可作为后续扩展（如存档分数、串口调试日志）。

---

8. 相关文档

文档	内容
`DOC/中文字模生成指南.md`	字模生成参数、接入步骤、常见踩坑、缺字清单
`DOC/磁力计参考算法/`	HMC5883L 参考驱动
`DOC/屏幕参考驱动/`	GC9A01 参考驱动

文档	内容
DOC/pic/	硬件照片与各应用/游戏截图